



第十三章 软件架构度量和评估

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

第13章 软件架构度量和评估

- 13.1 引言
- 13.2 典型的软件架构度量和评估方法
- 13.3 软件架构度量和评估过程
- 13.4 软件架构演化度量和评估实践
- 13.5 小结

13.1 引言 (I)

- 度量

度量 (measurement/metrics) 对于所有科学领域的进步都是至关重要的。在软件工程领域中，软件度量是保证软件质量的重要技术之一。

软件架构作为软件开发过程中早期阶段的设计模型，如果通过软件架构度量能够预测待开发的软件产品质量，并能够即时地发现早期设计缺陷，这对于减少开发风险和提高最终软件产品质量是非常重要的。

13.1 引言 (2)

- 以软件架构作为度量对象的原因
 - (1) 从架构出发，可以更有效地把握整个软件系统的设计结构和版本更新问题。
 - (2) 基于架构的度量可以在设计早期发现软件架构的不合理之处。
 - (3) 基于架构的度量可以对架构的演化程度进行评估和预测。
 - (4) 架构度量是一种静态的度量方法，不需要执行软件或者进行仿真，具有成本低、容易实现且不依赖于程序特定的运行环境的优点。

13.1 引言 (3)

- 单版本的软件架构度量和评估
 - 单个版本的架构度量技术一般会涉及如下两个活动：
 - (1) 定义架构评估框架，包括待度量的质量属性以及相应的度量指标。
 - (2) 针对每个度量指标设计相应的度量方法，并根据待测度量指标以及相应度量方法从架构设计文档中获取度量信息并计算。

13.1 引言 (4)

- 多版本的软件架构度量和评估
 - 可采用软件演化分析技术
 - 如：基于模型的演化评估方法、基于版本管理信息的演化评估方法、基于代码的演化评估方法等
 - 这些方法的基本步骤都是首先获取各个版本的分析模型，然后确定不同版本模型中元素之间的对应关系，发现相互之间的基本差异，然后以此为基础得到高层的演化模式或趋势分析结果。

13.2 典型的软件架构度量和评估方法 (I)

- 软件架构评估模型(Software Architecture Evaluation Model, SAEM)
- 软件架构评估信念网 (Software Architecture Assessment Belief Network, SAABNet)
- 基于结构指标的软件架构变更度量(Metrics of Software Architecture Changes based on Structural Metrics, SACMM)
- 软件架构静态评估(Static Evaluation of Software Architecture, SASAM)
- 架构级风险分析方法 (Architecture Level Reliability Risk Analysis, ALRRA)
- 层次分析法(Analytic Hierarchy Process, AHP)
- COSMIC+UML方法
- Shannon-based方法

13.2 典型的软件架构度量和评估方法 (2)

- 软件架构评估模型SAEM

SAEM方法将软件架构看作一个最终产品以及设计过程中的一个中间产品，从外部质量属性和内部质量属性两个角度来阐述它的评估模型，为软件架构的质量评估创建一个基础框架。

其流程为：

- (1) 对待评估的质量属性进行规约建模，描述架构的内部和外部质量属性。
- (2) 为外部和内部的质量属性创建度量准则。
- (3) 评估质量属性，包括数据搜集、度量和结果分析三个活动。

13.3 软件架构度量和评估过程（I）

- 在软件架构度量和评估中，由于软件架构的表示方式（即建模方法）多种多样，可以获得用于架构度量和评估的数据不同，不同的利益相关者（stakeholders）的度量和评估目标也不同，从而形成了存在不同种类的软件架构度量和评估方法。
- 本节将结合可维护性（maintainability）和可靠性（reliability）两个质量属性，详细说明软件架构度量和评估过程。该过程包括质量属性选择、可维护性度量及评估以及可靠性度量及架构评估等阶段。

13.3 软件架构度量和评估过程 (2)

(1) 质量属性选择

- 选择软件架构可维护性和可靠性的主要原因是：
 - 可维护性和可靠性的度量结果为**客观**度量值，有利于度量和评估的客观性。目前针对可维护性度量和可靠性度量研究比较广泛，其所需的度量数据大多可从架构中提取。
 - 可维护性度量和可靠性度量的**视角不同**。可维护性度量从静态角度度量架构的组织结构和逻辑依赖关系，评估软件架构的逻辑复杂程度；可靠性从动态角度度量架构模块间的交互过程，对软件的风险概率进行预测。

13.3 软件架构度量和评估过程 (3)

(2) 软件架构可维护性度量及评估

可维护性度量基于COSMIC所提出的架构度量模型，将传统的面向对象的软件评估方法应用于架构级别，以架构的组件图为度量对象，评估高层次上的架构复杂程度。

13.3 软件架构度量和评估过程 (4)

可维护性度量流程

- 1) 获取待度量系统的组件图。
- 2) 解析组件图，获取度量所需数据。所需度量信息包括组件图数目 L ，外部组件数目 $totalN$ ，外部组件间的连接关系 $totalE$ ，以及各个组件的依赖出边数目 E 、依赖入边数目 X ，提供接口数 W 、使用接口数 R 、子组件数目 S 以及子组件依赖关系的邻接矩阵 $AdjMatrix$ 。
- 3) 利用步骤 (2) 解析的组件图信息分别对可维护性6个度量指标进行度量，存储并返回度量结果。
- 4) 对度量结果进行分析评估。

13.3 软件架构度量和评估过程 (5)

软件架构可靠性度量及评估

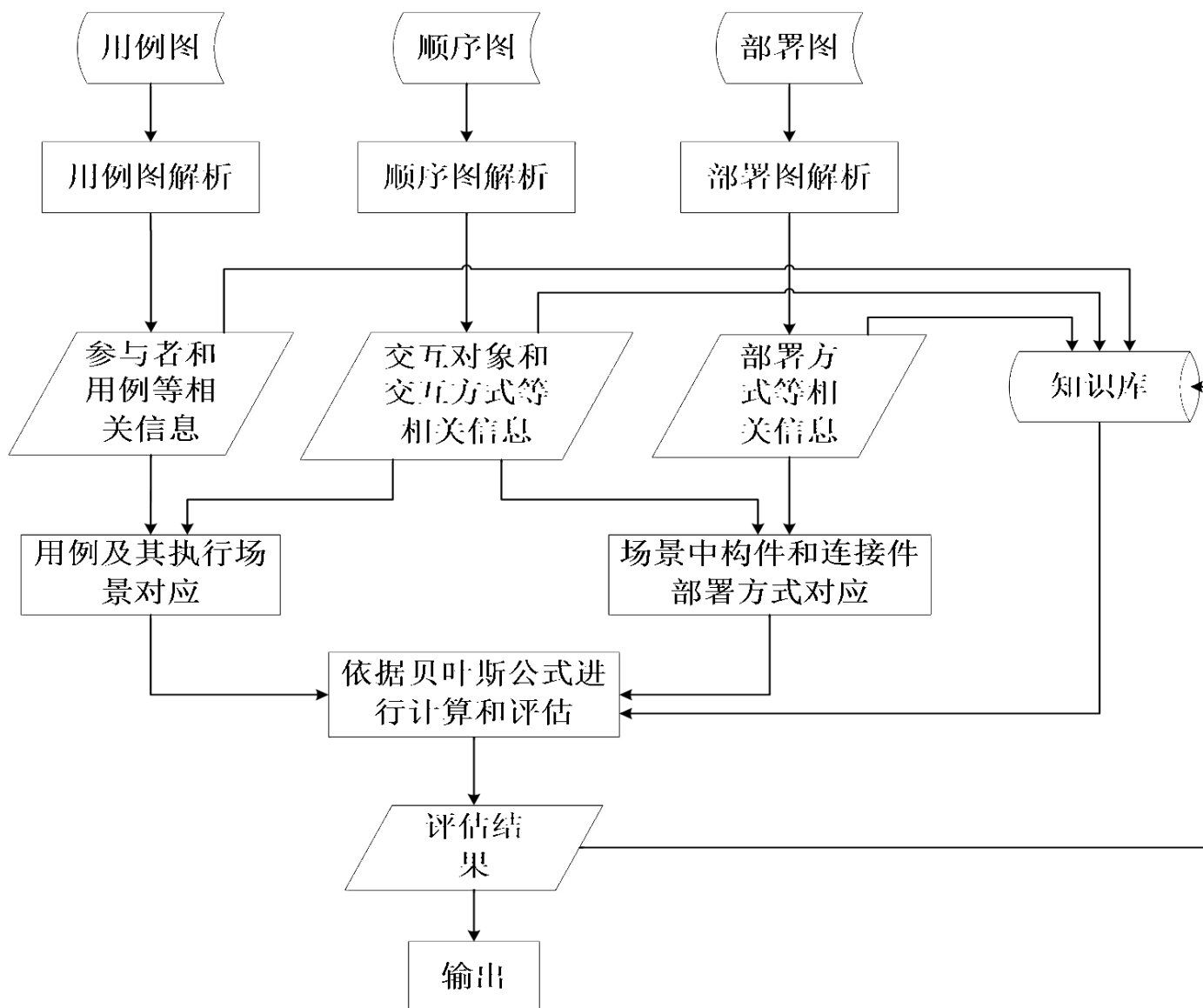
软件架构的可靠性被定义为软件架构的可靠概率RP (Reliability Probability) , RP的度量结果为一个实数值, 用以表示软件系统正常运行时软件架构不会发生错误的概率。

13.3 软件架构度量和评估过程 (6)

可靠性度量算法

- 计算每个场景的可靠概率
- 计算每个用例的可靠概率
- 计算每个用例被执行到的概率
- 计算整个系统可靠性
- 模型假设

13.3 软件架构度量和评估过程 (7)



13.3 软件架构度量和评估过程（示例）

SSAM 是基于架构可维护性的一项度量指标，而架构可维护性是基于 Common Software Measurement International Consortium (COSMIC)^[36]方法提出一种将传统的面向对象的软件评估方法应用于架构级别，以架构的构件图为度量对象，评估高层次上的架构复杂程度。

这里定义 SA-SMM 如下：

$$M_{\text{sa}} = \begin{cases} (TCC * LCC)^{\log_{10} CCN * FFC * CBO * RFC} & , TCC * LCC < 1 \\ 0.95^{\log_{10} CCN * FFC * CBO * RFC} & , TCC * LCC \geq 1 \text{ or } TCC * LCC = 0 \end{cases} \quad (2.1)$$

其中，CCN、FFC、CBO、RFC、TCC、LCC 为架构度量中可维护性的指标：

1. 圈复杂度（CCN，Cyclomatic Complexity Number）

根据图论和程序结构控制理论，以待测架构的构件图作为度量对象，表示构件图中由模块依赖关系组成的依赖控制路径的数量，公式 2.2 为圈复杂度计算方法。圈复杂度高的程序往往是最容易出现错误的程序，实践表明圈复杂度 $CCN \leq 10$ 为宜。

$$CCN = (\text{totalE} - \text{totalN}) + 2 * L \quad (2.2)$$

2. 扇入扇出度（FFC，Fan-in Fan-out Complexity）

扇入指直接调用该模块的上级模块的个数，扇入大表示模块的复用程序高。扇出指该模块直接调用的下级模块的个数。扇出越大，设计该模块时需要考虑的问题就越多，复杂性越高。对于设计良好的软件结构，通常顶层扇出比较大，中间扇出小，底层模块则有大扇入。公式 2.3 为架构扇入扇出度的计算方法。

$$FFC = CCN * ((E + W) * (X + R))^2 \quad (2.3)$$

13.3 软件架构度量和评估过程（示例）

The screenshot shows a software window titled "Architecture Evaluator - Project Import". It has a tabbed interface with four tabs: "Project Import", "Data Collection", "Maturity Measurement", and "Evolution Evaluation". The "Project Import" tab is active. Inside the window, there are three input fields: "Project name" and "Project version" are dropdown menus with the text "----Please select one----"; "Architecture documents" is a text box with a "Select..." button to its right. Below these fields is a "Check" button. Further down, a sequence of steps is listed: "Please operate according to following sequence :", "Architecture documents import", "Data collection", "SSAM,DSAM", and "ISAM". At the bottom, a note states: "At any time, you can use the Evolution evaluation function for adjacent version evolution."

Architecture Evaluator - Project Import

Project Import Data Collection Maturity Measurement Evolution Evaluation

Project name : ----Please select one----

Project version : ----Please select one----

Architecture documents : Select...

Check

Please operate according to following sequence :

Architecture documents import

▼

Data collection

▼

SSAM,DSAM

▼

ISAM

At any time, you can use the Evolution evaluation function for adjacent version evolution.

Architecture Evaluator - Maturity Measurement -> SSAM

Project Import Data Collection Maturity Measurement Evolution Evaluation

SSAM : 0.9283665

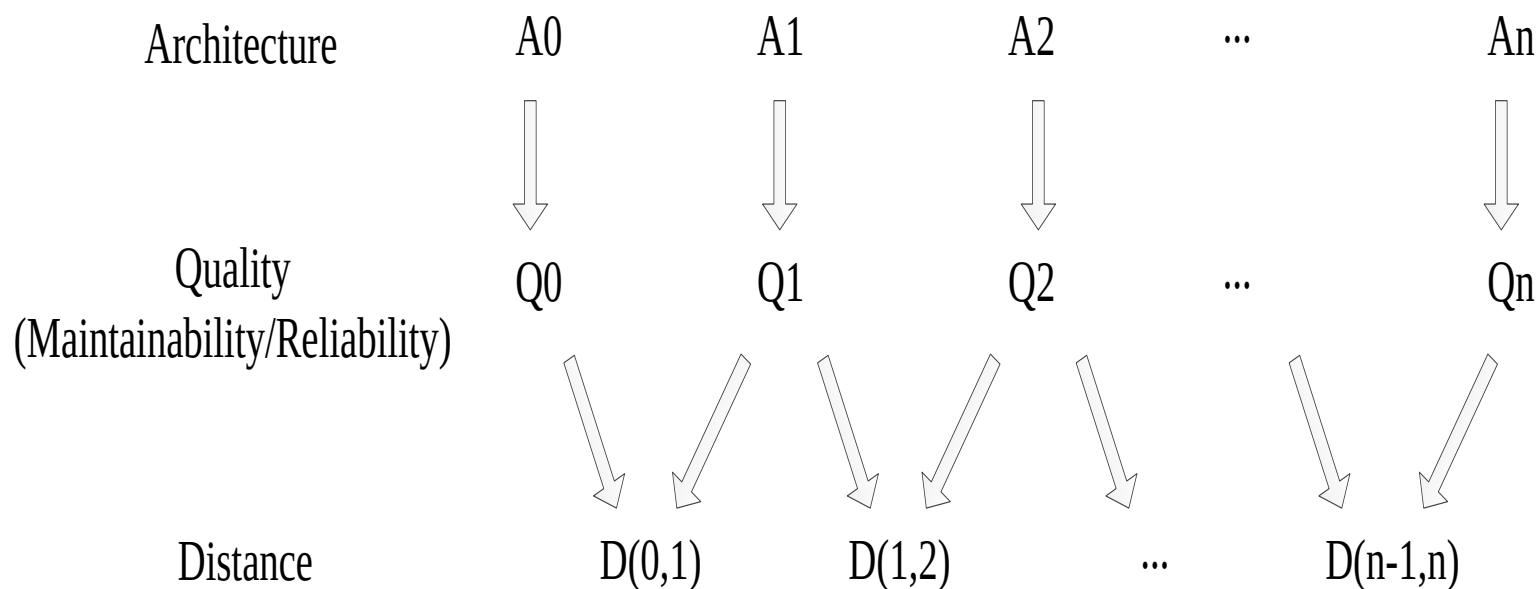
Evaluate SSAM and Save Data

Note:
SSAM: Static Software Architecture Maturity;
DSAM: Dynamic Software Architecture Maturity;
ISAM: Integrated Software Architecture Maturity

13.4 软件架构演化度量和评估实践

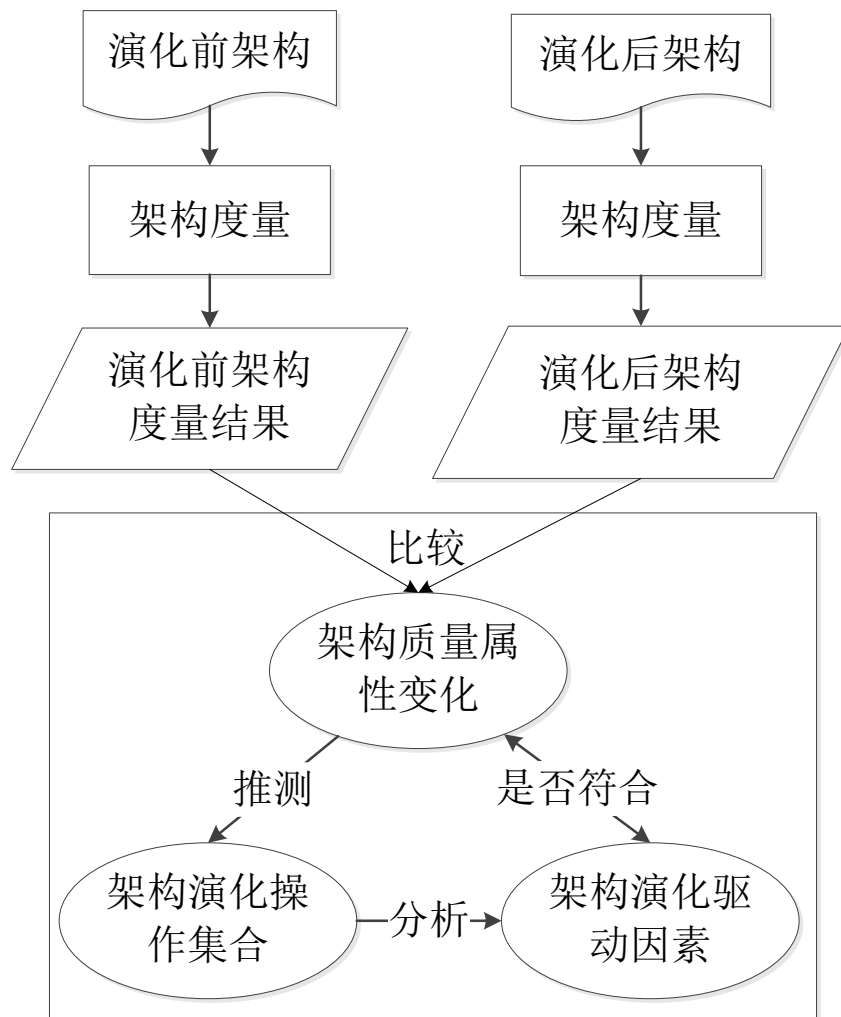
(1) 演化过程已知的软件架构演化

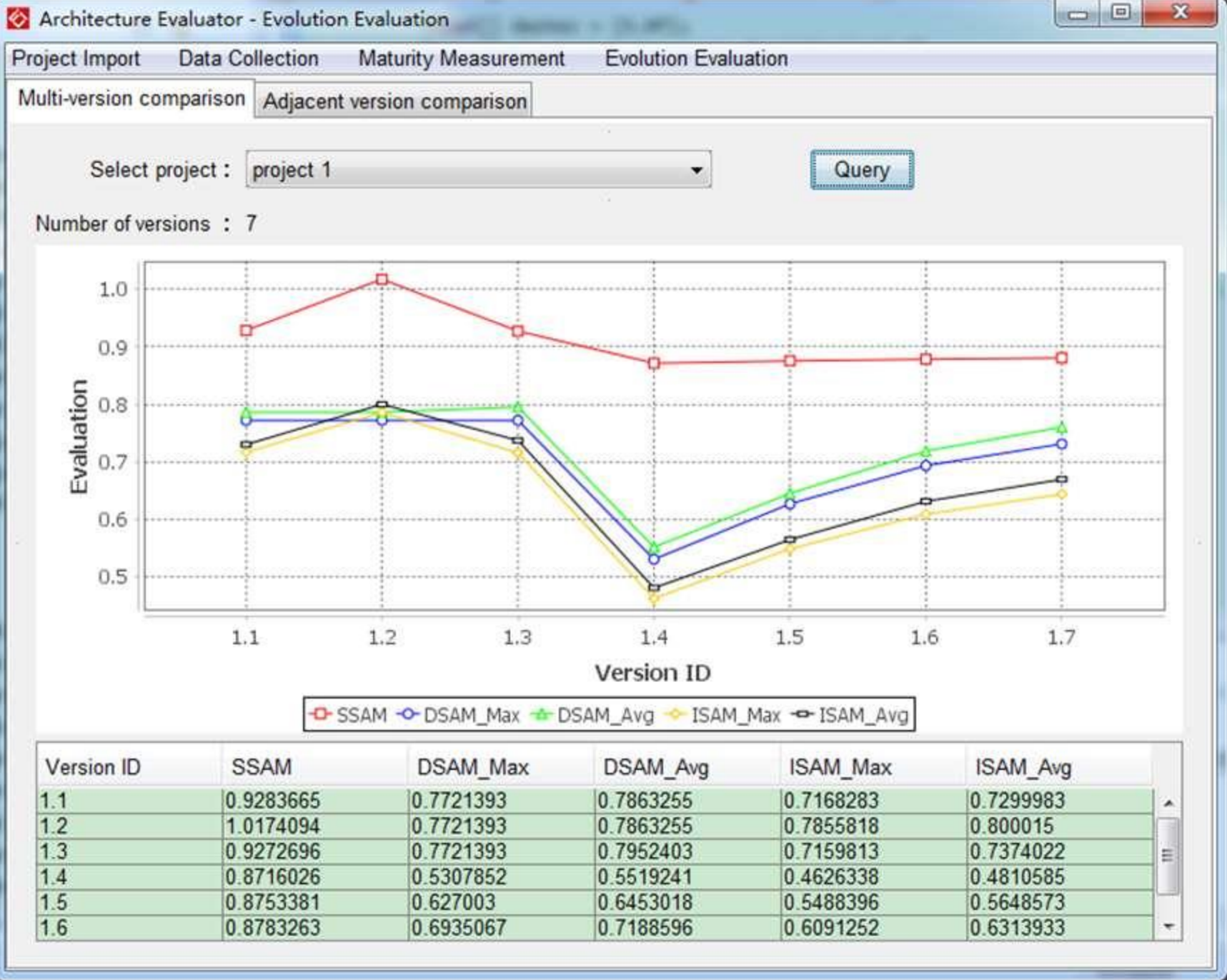
- 评估流程



13.4 软件架构演化度量和评估实践

(2) 演化过程未知的软件架构演化评估





Project name :

Software Architecture Tool

Project version :

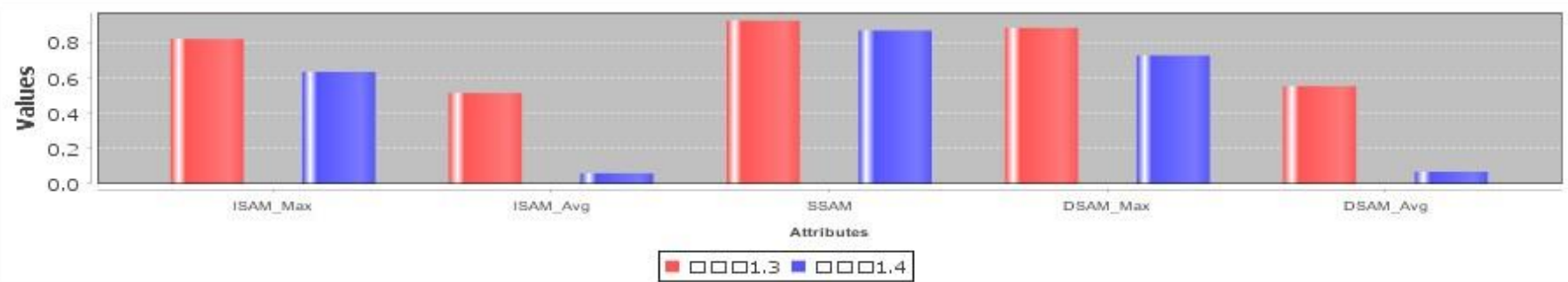
1.3

Direction :

☒ Backward☐ Forward

Compare

Maturity comparison :



Static Attributes						
VerID	CCN	FFC	CBO	RFC	TCC	LCC
1.3	4.0	13.0	0.44	1.0	1.0	1.0
1.4	8.0	61.0	0.53	2.0	1.0	1.0

Dynamic Attributes				
VerID	Reliability	CorrectRate	Performance_Max	Performance_Avg
1.3	1.0	0.67	0.2986744	1.4642465
1.4	1.0	0.5	0.4537493	3.8460965

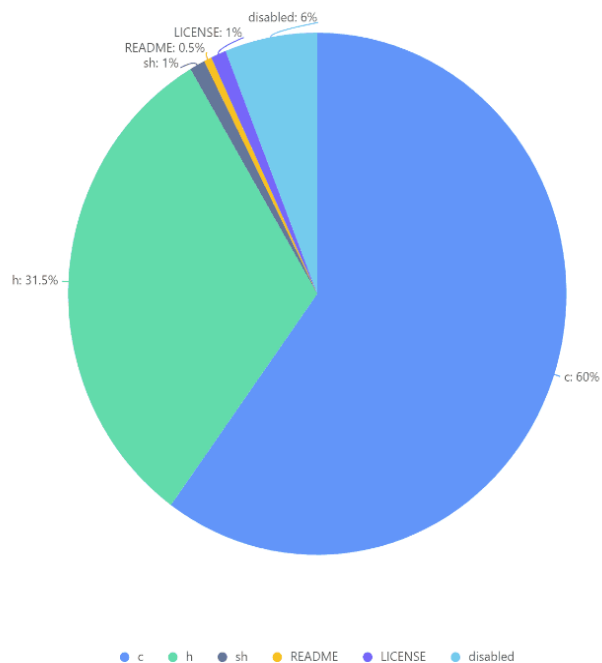
Performance Attributes								
VerID	MaxExcTime	AvgExcTime	MaxResTime	AvgResTime	MaxMemUse	MemUseUnitTime	MaxCPUUse	CPUUseUnitTime
1.3	35476.0	35476.0	35138.0	3942.0	110168.0	3.0	62.0	0.0
1.4	95529.0	95529.0	60053.0	8684.0	111188.0	3.0	79.0	0.0

补充：基于代码恢复的架构度量工具

度量流程

- 1) 获取待度量系统的代码。
- 2) 代码解析，获取度量所需数据。
- 3) 利用解析所得数据分别对相应的架构质量属性指标进行度量，存储并返回度量结果。
- 4) 对度量结果进行分析评估。

补充：基于代码恢复的架构度量工具



属性	数量
文件	200
函数	337
代码行	7420行

代码情况	数量	占比
有效代码行	6667行	89.85%
注释行	75行	1.01%
空行	678行	9.13%

补充：基于代码恢复的架构度量工具

函数统计

最大函数 (declspecs)

199行
平均行数
16行
最小行数
0行

文件统计

最大文件 (parse.c)

397行
平均行数
11行
最小行数
0行

文件与函数

函数最多的文件 (parse.c)

90
平均个数
2
最小行数
0

最大入度的函数

errorposf

22
平均入度
1
最小入度
0

出度最大的函数

expr

17
平均出度
1
最小出度
0

最大圈复杂度的文件

parse.c

397行
平均行数
11行
最小行数
0行

补充：基于代码恢复的架构度量工具

选择文件  请输入									
函数名	返回值	参数数量	变量数	IF语句	WHILE语句	SWITCH语句	入度	出度	代码行数
PlaceId	boolean	1	2	0	0	0	1	0	10
PleaseBody	void	3	1	0	0	2	1	1	5
PlaceHole	int	1	1	2	1	0	0	1	32
PlaceBody	void	3	3	1	0	0	0	1	12
NameId	String	2	3	1	2	1	1	0	22
getName	String	2	2	1	1	2	1	1	23
getPlace	String	1	2	0	0	0	1	0	10
getBody	int	3	1	0	0	2	1	1	5
setName	void	1	1	2	1	0	0	1	32
setPlace	void	3	3	1	0	0	0	1	12
ChangeNode	int	2	3	1	2	1	1	0	22

补充：基于代码恢复的架构度量工具

指标名称	度量参数	度量算法
可修改性	Out_i : 函数i的出度 In_i : 函数i的入度 N : 设计中函数的数量	$X = 1 - \frac{\sum_{i=1}^N \frac{Out_i + In_i}{N}}{N}$
可扩展性	I : 设计中API的个数 N : 设计中函数总数	$X = \frac{I}{N}$
易测试性	Out_i : 函数i的出度 N : 设计中函数的数量	$X_2 = 1 - \frac{\sum_{i=1}^N \frac{Out_i}{N}}{N}$
可替换性	R_i : 函数i的可替换性 In_i : 函数i的入度 N : 函数的数量	$X = \frac{\sum_{i=1}^N R_i}{N}$ $R_i = \frac{1}{1 + In_i}$
易理解性	$Comments$: 注释行 N : 有效代码行	$X = \frac{Comments}{N}$

补充：基于代码恢复的架构度量工具

指标名称	指标解释	度量方法	度量值
易理解性	易理解性度量的是开发人员对于设计和现有工程文件理解的难易程度，值越高，设计和工程文件越易于理解，越利于开发人员开展演进活动。	易理解性与模块中注释的标注有关系。模块中注释标注的越完善，占模块总数的比例越高，开发人员对模块的理解越容易，设计的易理解性越高。调用关系越复杂，设计修改难度越大。	0.8
可修改性	该指标度量的是对设计进行修改的难度，在本模型中，特指对设计改正和改进活动实施的难度，值越高，实施难度越低。	可修改性要与设计中各模块间的依赖关系复杂程度有关系，依赖关系越复杂，设计修改难度越大	0.751
可替换性	该指标度量设计中的模块被具有相似功能的模块替换的难度，以及删除特定功能模块的难度，值越高，替换/删除难度越小。	一个函数被其他函数调用的越多，替换或删除该函数造成的影响越大，替换或删除的难度也越大。函数被其他函数调用的关系反映在函数的入度，函数的入度越高，可替换性越低。	0.3481
可拓展性	该指标度量的是对设计进行扩展的难度，值越高，扩展难度越低。	在对设计进行扩展时，通常是基于现有API方法进行操作，所以该指标的度量与API方法占方法总数的比例有关系，该比例越高，则设计的可扩展性越高。	0.1
易测试性	该指标度量对该软件设计进行测试的难度，值越高，测试难度越小。	在对软件进行测试时，如果一个函数依赖于其他函数，通常需要将其一同测试，这样增加了测试的难度，因此该指标的度量与每个函数的出度直接。相关	0.92

13.5 小结

- 本章首先详细介绍了软件架构度量和评估的各种方法。其优势在于可以在设计早期发现架构相关质量问题，并基于评估结果对软件架构进行预测。
- 将基于度量的软件架构评估方法应用于软件演化，通过对演化前后不同版本的架构进行度量，分析相关质量属性在每次演化之后的变化情况，找出架构内部结构的变化与外部质量属性变化间的关联。
- 通过评估演化过程中多个版本的架构，归纳其变化趋势，达到在总体上把握和控制软件演化过程的目的。



THANK YOU